SYSTEM DESIGN & ARCHITECTURE SPECIFICATION

CLABTOPro

C-Laboratory Online Judging & AI Tutoring Platform

Comprehensive technical blueprint detailing high-level system architecture, dual-engine sandboxed execution, non-authoritative AI tutoring, concurrency bounds, real-time proctoring telemetry, and database models.

PROJECT REPOSITORY
CLABTOPro / Major_Project

PRIMARY BACKEND
Render Web Service (Node 20 + GCC 12)

SECURITY MODEL
Piston Docker + Local Sandboxed GCC

DOCUMENT VERSION
1.0.0 (Production Architecture)

SECTION 1

Executive Summary & System Objectives

CLABTOPro is designed to solve the critical challenges of conducting high-enrollment C programming laboratories (~120 concurrent students). Traditional automated judging systems either rely on vulnerable host compilation or lack intelligent pedagogical feedback. CLABTOPro bridges this gap with deterministic judging coupled with decoupled AI diagnostics.

1.1 Core Functional Requirements

• **Interactive In-Browser IDE:** Web-based coding environment with standard C11 support, input simulation, and instant execution.

• **Deterministic Online Judging:** Automated verification against sample and hidden test cases with millisecond execution accounting.

• **Dual Execution Sandboxing:** Primary execution inside containerized Piston instances with seamless fallback to isolated local GCC environments.

• **AI Failure Assistant:** Automatic Socratic guidance on compiler, runtime, and logic errors without leaking full solutions.

• **Anti-Cheat Proctoring Telemetry:** Real-time tracking of browser focus, tab switches, and window blur events attached to submissions.

• **Role-Based Management & Progress:** Dual roles (Student vs Admin/Teacher), daily streaks, topic completion tracking, and aggregate performance metrics.

1.2 Non-Functional Requirements & Design Principles

• **100% Deterministic Judging:** Verdicts are decided strictly by normalized I/O comparison. AI has zero authority over pass/fail decisions.

• **Fault-Tolerant High Availability:** Zero downtime if external Docker or AI APIs degrade, via automatic dual-engine execution and multi-model AI cascading.

• **Bounded Concurrency:** Strict queue limits (max concurrency = 4) preventing server CPU/memory exhaustion during deadline spikes.

• **Information Confidentiality:** Hidden test case inputs and expected outputs are strictly sanitized before returning wire payloads to students.

Architectural Invariant: Decoupled AI Authority

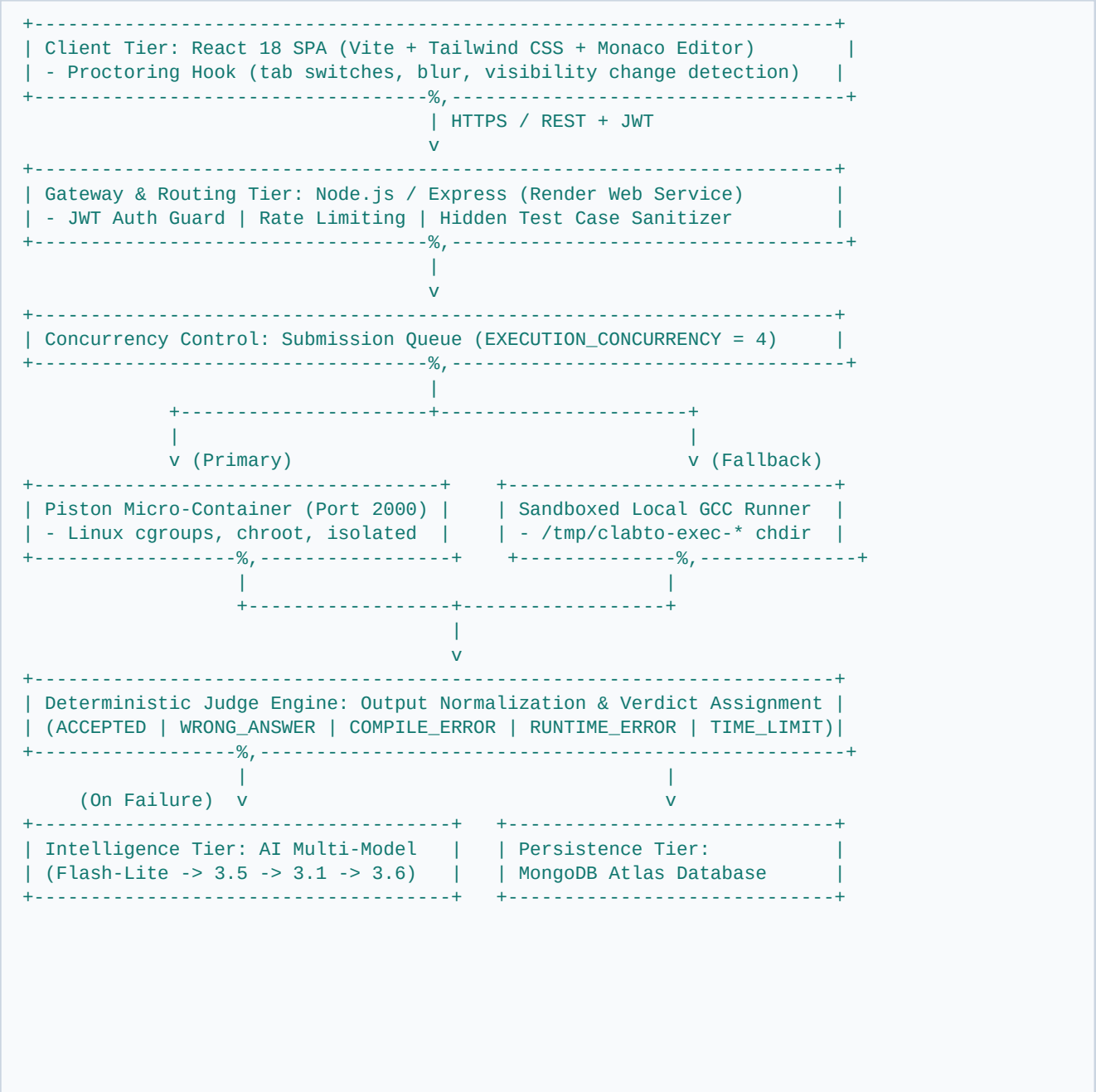
The AI evaluation layer is strictly non-authoritative. It is invoked purely as an asynchronous advisor when a submission fails, ensuring grading integrity is 100% reproducible and tamper-proof.

SECTION 2

High-Level Architecture (HLD)

The system utilizes a 5-tier decoupled architecture separating client rendering, gateway control, execution scheduling, intelligence advisory, and persistent storage.

2.1 Architectural Flow Diagram



2.2 Layer Responsibilities

• **Client Tier:** Delivers responsive UI, manages code editor state, captures anti-cheat signals, and displays sanitized execution results.

• **API Gateway:** Authenticates sessions, validates request formats, enforces role guards, and sanitizes output payloads.

• **Execution Tier:** Regulates compilation throughput, enforces memory and CPU limits, and calculates

deterministic verdicts.

- **Intelligence Tier:** Provides pedagogical hints for failed submissions using resilient fallback AI models.
- **Persistence Tier:** Stores users, problems, test cases, submissions, and student streaks in MongoDB Atlas.

SECTION 3

Execution Engine & Sandboxing Deep Dive

Executing untrusted student code requires strict sandboxing, timeout enforcement, and memory caps. CLABTOPro achieves this through a dual-engine architecture with automatic failover.

3.1 Dual-Engine Strategy

• **Primary Engine (Piston):** Interacts with an isolated Piston micro-container via REST API. Employs Linux namespaces, cgroups, user isolation, and chroot.

• **Fallback Engine (Local GCC):** Compiles and executes locally inside `/tmp/clabto-exec-<uuid>/`. Ideal for cloud environments (Render) where Docker daemon access is restricted.

3.2 Safety & Sandboxing Mechanisms

• **Compilation Constraints:** Compiles with: gcc -O2 -Wall -std=c11 solution.c -o solution -lm. Flags prevent UB and enforce standard semantics.

• **Wall-Clock Timeouts:** Child process execution is wrapped with a 3,000 ms timer. Exceeding limits triggers SIGKILL to stop infinite loops.

• **Stream Error Protection:** Attaches child.stdin.on('error') handler to silently intercept broken pipes (EPIPE) when student binaries terminate early.

• **Ephemeral Workspace Reclaim:** Guaranteed cleanup in a finally block removes temporary compilation artifacts and binaries from disk.

3.3 Bounded Concurrency Queue

To handle burst loads when a lab of 120 students submits simultaneously, submissionQueue.js limits active concurrent compilation/run tasks to 4. Subsequent requests are queued in FIFO order, preventing host CPU starvation.

SECTION 4

AI Tutoring & Multi-Model Cascade

The AI tutoring service provides immediate educational assistance when submissions fail. To ensure 100% uptime during Google Cloud capacity spikes, a cascading multi-model fallback chain is enforced.

4.1 Multi-Model Failover Architecture

```
Request -> [gemini-flash-lite-latest] --(503 Spike)--> [gemini-3.5-flash-lite]
                                                                | (503 Spike)
                                                                v
Response <- [Deterministic Regex Explainer] <--(Fail)-- [gemini-3.1-flash-lite]
                                                                | (503 Spike)
                                                                v
                                                       [gemini-3.6-flash]
```

4.2 Pedagogical Guardrails

• **No Direct Solution Giving:** Prompts explicitly instruct the model to provide conceptual hints rather than rewritten code blocks.

• **Evidence-Based Inputs:** AI receives compiler errors, failed sample test mismatches, and execution signals, ensuring targeted feedback.

• **Rule-Based Fallback:** If all external AI APIs fail or network breaks, a local regex-based diagnostic engine generates structured hints for common syntax errors and segfaults.

SECTION 5

Proctoring, Security & Anti-Cheat

Academic integrity is maintained through multi-layered telemetry and confidential test case sanitization.

5.1 Client-Side Proctoring Telemetry

• **Viewport Focus Tracking:** Listens to visibilitychange and window.onblur events. Any navigation away from the lab IDE increments tabSwitches.

• **Warning Modal System:** Displays immediate feedback when focus is lost, warning students that events are audited.

• **Submission Auditing:** Every code submission permanently records the accumulated tab switch count and timestamps into MongoDB for instructor review.

5.2 Hidden Test Case Sanitization

Zero-Leakage Confidentiality Guarantee

Student responses are filtered through sanitizeSubmissionForStudent(). Hidden test case inputs, expected outputs, and diffs are completely stripped before serialization.

SECTION 6

Data Architecture & Schema Design

The database schema in MongoDB Atlas is optimized for fast lookups, analytics, and audit logging.

6.1 Core Entities

• **User:** Stores credentials (bcrypt), email, role ('student' | 'admin'), and aggregate problems solved.

• **Problem:** Defines statement, difficulty, time/memory limits, sample test cases (public), and hidden test cases (confidential).

• **Submission:** Contains submitted code, deterministic verdict, test breakdown, execution time, AI hints, and tabSwitches audit.

• **Progress:** Tracks daily active streaks (currentStreak, maxStreak) and list of solved problem IDs.

6.2 Data Schema Summary

```
User:       { _id, name, email, password, role: 'student'|'admin', problemsSolved }
Problem:    { _id, title, description, difficulty, sampleTests: [], hiddenTests: [] }
Submission: { _id, userId, problemId, code, status, passedTests, totalTests, tabSwitches,
aiFeedback }
Progress:   { _id, userId, currentStreak, maxStreak, lastActiveDate, solvedProblems: [] }
```

SECTION 7

Deployment Topology & Operations

CLABTOPro is deployed on modern cloud infrastructure providing containerized isolation without requiring complex Kubernetes management.

7.1 Production Environment Topology

• **Backend (Render):** Hosted as a Dockerized Web Service (Node 20 + GCC 12/13). Includes built-in compiler toolchain for zero-dependency execution.

• **Database (MongoDB Atlas):** Managed cloud cluster with automated daily snapshots, connection pooling, and replica sets.

• **Frontend (Vercel):** Edge-distributed static hosting with automatic SPA rewrite rules and SSL termination.

• **External AI (Google Gemini):** Secure HTTPS API communication with multi-model failover keys.

7.2 Operational Health Endpoints

• **Execution Health (/api/execution/health):** Returns live status of compilation engines, active compiler version, concurrency limits, and queue load.

• **Admin Metrics (/api/admin/overview):** Provides real-time submission pass rates, error distributions, and proctoring audit flags.